Sem Repository (Errado)

O código de persistência está espalhado pela aplicação.

```
public class ProductService
{
    private readonly SqlConnection _connection;

    public ProductService(string connectionString)
    {
        _connection = new SqlConnection(connectionString);
    }

    public List<string> GetAllProductNames()
    {
        var productNames = new List<string>();
        _connection.Open();

        using var command = new SqlCommand("SELECT Name FROM Products",
_connection);
        using var reader = command.ExecuteReader();
        while (reader.Read())
        {
            productNames.Add(reader.GetString(0));
        }

        _connection.Close();
        return productNames;
    }
}
```

Problemas:

- O `ProductService` depende diretamente do banco de dados.
- Alterações no banco de dados exigem alterações na lógica de negócios.

Com Repository (Certo)

O acesso a dados é encapsulado no repositório.

```
// Entidade
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}

// Interface do Repositório
public interface IProductRepository
{
    IEnumerable<Product> GetAllProducts();
    Product GetProductById(int id);
    void AddProduct(Product product);
    void DeleteProduct(int id);
}

// Implementação do Repositório
public class ProductRepository : IProductRepository
{
    private readonly SqlConnection _connection;

    public ProductRepository(string connectionString)
    {
        _connection = new SqlConnection(connectionString);
    }

    public IEnumerable<Product> GetAllProducts()
    {
        var products = new List<Product>();
        _connection.Open();

        using var command = new SqlCommand("SELECT Id, Name, Price FROM
Products", _connection);
        using var reader = command.ExecuteReader();
        while (reader.Read())
        {
            products.Add(new Product
            {
                Id = reader.GetInt32(0),
                Name = reader.GetString(1),
                Price = reader.GetDecimal(2)
            });
        }
```

```
        _connection.Close();
        return products;
    }

    public Product GetProductById(int id)
    {
        _connection.Open();

        using var command = new SqlCommand("SELECT Id, Name, Price FROM
Products WHERE Id = @Id", _connection);
        command.Parameters.AddWithValue("@Id", id);

        using var reader = command.ExecuteReader();
        if (reader.Read())
        {
            var product = new Product
            {
                Id = reader.GetInt32(0),
                Name = reader.GetString(1),
                Price = reader.GetDecimal(2)
            };

            _connection.Close();
            return product;
        }

        _connection.Close();
        return null;
    }

    public void AddProduct(Product product)
    {
        _connection.Open();

        using var command = new SqlCommand("INSERT INTO Products (Name,
Price) VALUES (@Name, @Price)", _connection);
        command.Parameters.AddWithValue("@Name", product.Name);
        command.Parameters.AddWithValue("@Price", product.Price);
        command.ExecuteNonQuery();

        _connection.Close();
    }

    public void DeleteProduct(int id)
    {
        _connection.Open();

        using var command = new SqlCommand("DELETE FROM Products WHERE Id
= @Id", _connection);
```

```
        command.Parameters.AddWithValue("@Id", id);
        command.ExecuteNonQuery();

        _connection.Close();
    }
}

// Uso do Repositório na Lógica de Negócios
public class ProductService
{
    private readonly IProductRepository _repository;

    public ProductService(IProductRepository repository)
    {
        _repository = repository;
    }

    public void PrintAllProducts()
    {
        var products = _repository.GetAllProducts();
        foreach (var product in products)
        {
            Console.WriteLine($"{product.Id}: {product.Name} -
{product.Price:C}");
        }
    }
}

// Exemplo de uso
var repository = new ProductRepository("your_connection_string");
var service = new ProductService(repository);
service.PrintAllProducts();
```

Vantagens do Exemplo Certo

1. **Desacoplamento**: A lógica de negócios (`ProductService`) não depende de como os dados são persistidos (`ProductRepository`).
2. **Substituição**: Fácil trocar o mecanismo de persistência (ex.: trocar SQL Server por MongoDB).
3. **Testes Unitários**: A interface `IProductRepository` pode ser mockada para testes.